



Original Article

Leveraging physics-informed neural computing for transport simulations of nuclear fusion plasmas

J. Seo^{*}, I.H. Kim, H. Nam

Department of Physics, Chung-Ang University, Seoul, South Korea

ARTICLE INFO

Keywords:

Nuclear fusion

Tokamak transport

Physics-informed neural network

ABSTRACT

For decades, plasma transport simulations in tokamaks have used the finite difference method (FDM), a relatively simple scheme to solve the transport equations, a coupled set of time-dependent partial differential equations. In this FDM approach, typically over $O(10^5)$ time steps are needed for a single discharge, to mitigate numerical instabilities induced by stiff transport coefficients. It requires significant computing time as costly transport models are repeatedly called in a serial manner, proportional to the number of time steps. Additionally, the unidirectional calculations of FDM make it difficult to predict regions prior to the initial condition or apply additional temporal constraints. In this study, we discuss using a new scheme to solve plasma transport based on physics-informed neural networks (PINNs). PINN iteratively updates a function that maps spatiotemporal coordinates to plasma states, gradually reducing errors in transport equations. The required number of updates in PINNs is several orders of magnitude less than the chronological iterations in FDM. Furthermore, it is free from numerical instabilities arising from finite grids and enables more versatile semi-predictive simulations with arbitrary spatiotemporal constraints. In this paper, we discuss the features and potentials of the tokamak transport solver using PINNs through comparisons with FDM, and also its drawbacks and challenges.

0. Introduction

The tokamak is one of the most promising concepts for nuclear fusion reactors, and research for stable and efficient energy production based on nuclear fusion is actively underway, including the ITER international science project [1]. To analyze and predict the fusion performance and plasma confinement in tokamaks, it is essential to calculate the transport of fusion plasma. Calculating plasma transport in tokamaks requires solving a multi-variate coupled set of time-dependent, nonlinear partial differential equations (PDEs) in torus geometry. Fortunately, the three-dimensional torus geometry can be reduced to a two-dimensional problem through the axisymmetric assumption. Also, since plasma has the property of being “frozen in magnetic flux” in the low resistivity limit, it can be approximated as a one-dimensional problem by using the magnetic flux coordinate (ρ), so-called 1.5D transport solving. Eqs. (1) and (2) show a set of PDEs for particle and heat transport of plasma [2]. Here, V and V' are the plasma volume and its spatial derivative, and Φ and $\dot{\Phi}$ are the toroidal magnetic flux and its temporal derivative, which are geometric functions for a given magneto-hydrodynamic equilibrium. n_s and T_s are the unknown density and temperature of the species s , where $s = e$ and i denote the electron and ion, respectively. S_s and P_s are the particle source and power, and

Γ_s and q_s are the particle and heat flux, which can be represented as Eqs. (3) and (4). D and χ_s are the particle and heat diffusion coefficients, respectively. If D and χ_s are determined by a transport coefficient model, Eqs. (1)–(4) form a closed set of equations that can determine the unknowns, n_s and T_s . Note that this is an example set of plasma transport equations, and each transport solver uses different equations depending on assumptions and geometry [3,4].

$$F_{particle}(\rho, t) \equiv \frac{1}{V'} \left(\frac{\partial}{\partial t} - \frac{\dot{\Phi}}{2\Phi} \frac{\partial}{\partial \rho} \right) (V' n_s) + \frac{1}{V'} \frac{\partial}{\partial \rho} \Gamma_s - S_s = 0 \quad (1)$$

$$F_{heat}(\rho, t) \equiv \frac{3}{2} \frac{1}{V'^{5/3}} \left(\frac{\partial}{\partial t} - \frac{\dot{\Phi}}{2\Phi} \frac{\partial}{\partial \rho} \right) (V'^{5/3} n_s T_s) + \frac{1}{V'} \frac{\partial}{\partial \rho} \left(q_s + \frac{5}{2} \Gamma_s T_s \right) - P_s = 0 \quad (2)$$

$$\Gamma_s = -DV' \langle (\nabla \rho)^2 \rangle \frac{\partial n_s}{\partial \rho} \quad (3)$$

$$q_s = -\chi_s V' \langle (\nabla \rho)^2 \rangle n_s \frac{\partial T_s}{\partial \rho} \quad (4)$$

^{*} Corresponding author.

E-mail address: jseo@cau.ac.kr (J. Seo).

Conventionally, finite difference methods (FDM) have been predominantly used to solve these nonlinear coupled PDEs, such as in the traditional solvers including ASTRA [2,5], FASTRAN [6], and RAPTOR [7]. Many integrated tokamak simulation suites, such as TRANSP [8] and TRIASSIC [9], also utilize FDM-based transport solvers. In fact, calculating the transport equations given the diffusion coefficient has been a classical problem in fluid dynamics for decades. However, in the case of fusion plasma, the particle and heat transport are governed by multiple complex magneto-hydrodynamics and turbulence phenomena at different spatiotemporal scales [10]. These multiple physical mechanisms comprehensively determine the transport coefficients, D and χ_s in Eqs. (3) and (4). Since each phenomenon requires computation at different spatiotemporal scales, high-fidelity computation (ex. gKPSP [11]) covering most of them requires several days, and even reduced transport coefficient models like TGLF [12] and QuaLiKiz [13] require at least minutes for a single step calculation. Therefore, in many cases, the total time required for plasma transport simulation in tokamaks is dominated by the number of serial calls for these external models to calculate the coefficients.

When computing plasma transport using FDM, given initial plasma profiles ($n_e, T_e, T_i, \dots$) at a single time point, transport coefficients are first calculated for each spatial grid in ρ . Then, the profiles for the next time step are computed through the discretization of Eqs. (1) and (2), explicitly or implicitly. This process is illustrated in Fig. 1a. In this case, multiple calculations for the ρ grid are independent of each other, allowing for parallelization through multiple processors. However, serial calculations over time are difficult to parallelize since each step requires the complete profiles of the previous step. To prevent numerical instability, the time interval (Δt) must satisfy $\Delta t < 0.5(\Delta\rho)^2/\max(D \text{ or } \chi_s)$, where $\Delta\rho (= O(10^{-2})\rho_{\text{boundary}})$ is the spatial interval for discretization [14]. Thus, typically $O(10^5)$ serial iterations are required for a transport simulation of a single tokamak discharge. This takes a large computation time with desktop-level computers, even when using reduced transport models. In addition, FDM inevitably requires a complete set of plasma profiles for initial conditions. In actual experiments, it is often difficult to measure the plasma state at the discharge beginning due to the validity regime of the diagnostic systems, and each diagnostic system for a different profile measures at a different time point. Therefore, (i) we need to interpolate the different plasma profiles to the same time point to get a complete set of profiles, which loses the fidelity, and (ii) it is difficult to predict the plasma states before the initial measurements, such as the ramp-up phase.

In this study, we propose a novel plasma transport simulation workflow that can significantly reduce the computation time cost by eliminating time-ordered serial computations, using physics-informed neural network (PINN) computing. As shown in Fig. 1b, PINN can be used to iteratively update complete spatiotemporal profiles simultaneously, finding a solution function ($u = \theta(\rho, t)$) that minimizes the error in the PDEs. Here, the number of serial computations required for iterative updates for PINN (N_{iter}) is usually less than the number of time-ordered serial computations for FDM ($N_t = O(10^5)$). Moreover, since PINN allows to use arbitrary spatiotemporal constraints instead of a complete set of initial profiles, predictive simulations can be performed with more preservation of the fidelity of diagnostics from experiments than FDM.

1. Architecture and losses for PINN

Recently, deep learning and neural networks have been widely applied to modeling and optimization problems in tokamaks [15–19]. The physics-informed neural network (PINN) is a class of deep learning models that incorporate the governing physics, typically represented as PDEs, into a neural network architecture [20–22]. By constraining the neural network to satisfy these governing equations, PINNs are capable of generating solutions that not only fit the observed data such as initial and boundary conditions but also adhere to the underlying physics.

The incorporation of physical constraints in PINN-based simulations has offered a promising alternative to traditional numerical methods in fluid and plasma physics [23–26].

In particular in plasma transport simulation, due to the “stiff” transport coefficients (D, χ_s) that are highly sensitive to the gradients and higher-order spatial derivatives of the profiles, the FDM often induces strong numerical instabilities so that it requires a large number of iterations with very small time intervals. Technical methods to reduce these numerical instabilities in FDMs have been continuously developed [27,28]. On the other hand, the PINN method employed in this work can avoid these issues, not reduce them, and also potentially offer faster computation, which will be discussed later in detail.

The architecture of the PINN model to solve the transport equations is shown in Fig. 2. First, the PINN model takes spatiotemporal coordinate values (ρ, t) as input. Subsequently, the predicted plasma state ($\hat{u}$) is obtained through the computation of the multi-layer perceptron (θ). In this study, we set $\hat{u} = (n_e, T_e)$ as output variables; however, without loss of generality, we can extend the output to include additional transport variables such as T_i , magnetic flux, and momentum in the future. Calculating only electron density and temperature can be applied to electron-dominant heated plasmas, such as [29].

In this study, all instances of the neural network are designed with the same simple structure, consisting of three hidden layers, each with 20 nodes. Hyperbolic tangent activation is applied to each hidden layer to provide nonlinearity. We used the TensorFlow library [30] to build the network and SciPy [31] to optimize its internal parameters with the L-BFGS optimizer [32]. To optimize the PINN model, defining the objective function is necessary. Unlike data-driven deep learning, which defines the objective function as the error between the true and predicted values, the optimization of PINN minimizes the errors of the given PDEs and the constraints. The objective function is defined as a weighted sum of three terms: PDE loss (Eq. (5)), boundary condition (BC) loss (Eq. (6)), and initial condition (IC) loss (Eq. (7)).

$$L_{PDE}(\theta) = \frac{1}{N_{\text{sample}}} \sum_{j=1}^{N_{\text{sample}}} \left((F_{\text{particle}}(\rho_j, t_j))^2 + (F_{\text{heat}}(\rho_j, t_j))^2 \right) \Big|_{\theta} \quad \text{for } (\rho_j, t_j) \in \Omega \quad (5)$$

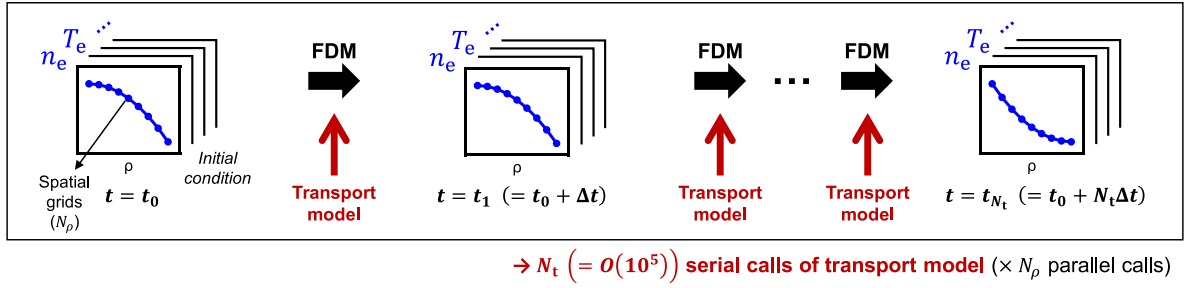
$$L_{BC}(\theta) = \frac{1}{N_{BC}} \sum_{j=1}^{N_{BC}} \left(\hat{u}(\rho_j, t_j) - u^{BC}(t_j) \right)^2 \Big|_{\theta} \quad \text{for } (\rho_j, t_j) \in \partial\Omega_B \quad (6)$$

$$L_{IC}(\theta) = \frac{1}{N_{IC}} \sum_{j=1}^{N_{IC}} \left(\hat{u}(\rho_j, t_j) - u^{IC}(\rho_j) \right)^2 \Big|_{\theta} \quad \text{for } (\rho_j, t_j) \in \partial\Omega_I \quad (7)$$

Here, Ω is the spatiotemporal domain of (ρ, t) and $\partial\Omega_B$ or I is the spatial and temporal boundaries of the domain. N_{sample} , N_{BC} , and N_{IC} are the number of samples on each domain for PDE, BC, and IC loss calculations, respectively. The sampled points are randomly distributed on each domain and are resampled every ten iterations. Detailed settings, including the actual Python scripts, can be found in our GitHub repository [33].

In each iteration shown in Fig. 2, the neural network calculates the output values $\hat{u}$ for randomly sampled points in the (ρ, t) domain through forward propagation. Since the neural network consists of differentiable nodes, it can easily calculate derivatives with respect to spatiotemporal coordinates. Note that these derivatives are exact derivatives, not approximated discretized values as in FDM. From the calculated output values and their derivatives, transport coefficients (D, χ_s) can be obtained using external transport models. Eq. (5) can be calculated based on the derivatives and transport coefficients, while Eqs. (6) and (7) can be computed using the output values in the IC and BC domains. Finally, θ is optimized to minimize the objective function, the weighted sum obtained from Eq. (8), where the

a. FDM: solving $u = \{u_\rho^t\}$



b. PINN: solving $u = \theta(\rho, t)$

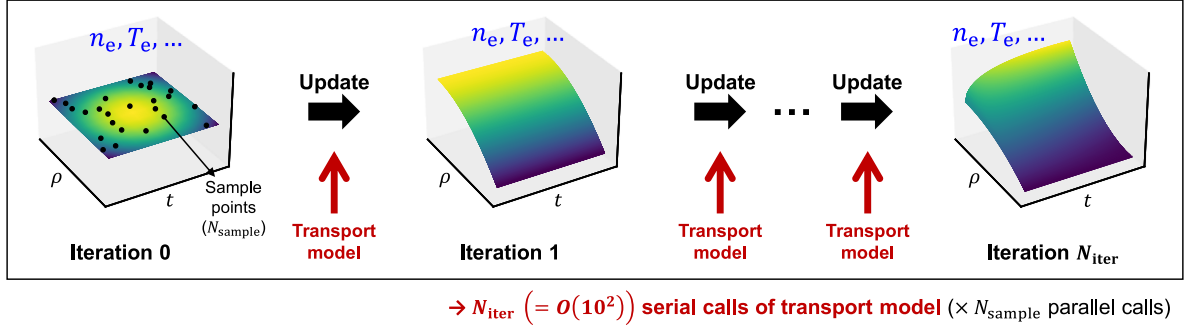


Fig. 1. Comparison of the serial iterations in FDM and PINN, two different approaches to numerically solving transport equations. a, The serial process using FDM. FDM calculates a set of plasma states on finite grids, $\{u_\rho^t\}$, through solving discretized equations incrementally by advancing Δt each. b, The serial process using PINN. PINN iteratively updates the function of spatiotemporal coordinates, θ , to minimize the error in the transport equations.

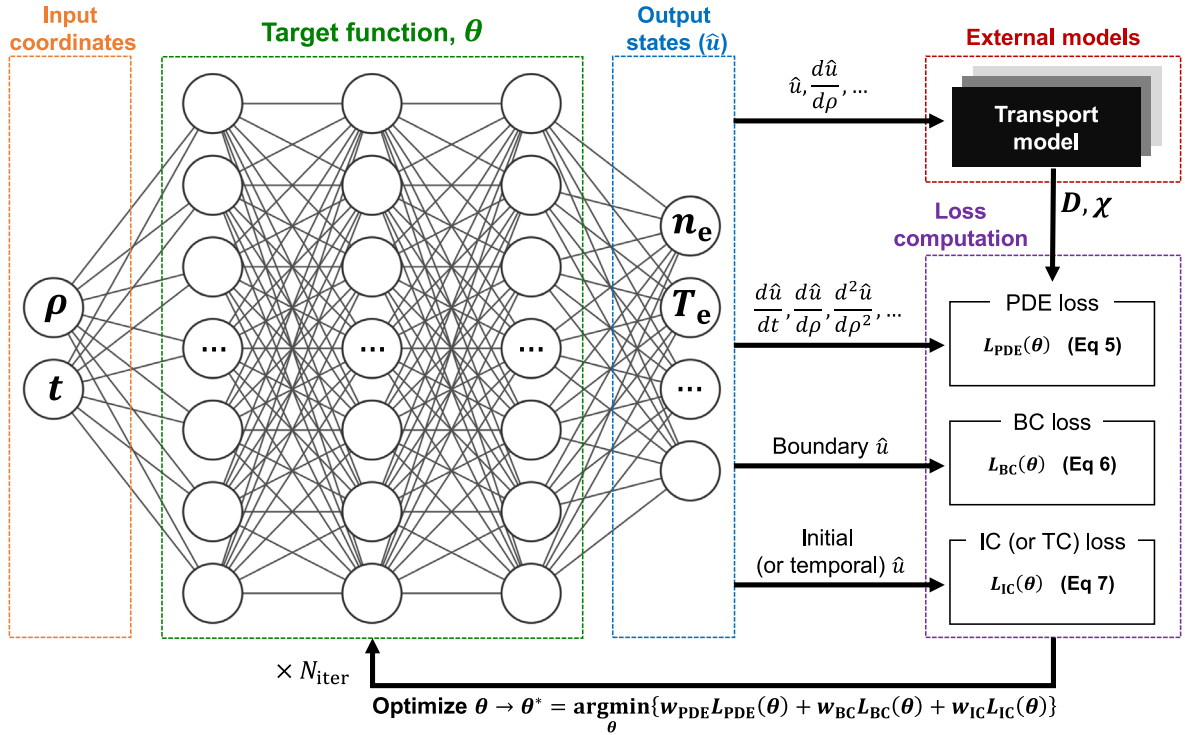


Fig. 2. Architecture of the neural network using physics-informed loss. Unlike data-driven deep learning, PINN's loss is calculated with the error in given physics equations. In the process of estimating the loss, coefficients required for the equations are obtained through external models such as transport models.

weights (w_{PDE}, w_{BC}, w_{IC}) are free parameters. The optimization of θ is described in Eq. (9).

$$L_{PINN}(\theta) = w_{PDE}L_{PDE}(\theta) + w_{BC}L_{BC}(\theta) + w_{IC}L_{IC}(\theta) \quad (8)$$

$$\theta \rightarrow \theta^* = \underset{\theta}{\operatorname{argmin}} \{L_{PINN}(\theta)\} \quad (9)$$

2. Predictive transport simulation using PINN

The purpose of predictive transport simulations is to compute the subsequent states of the plasma, given the initial conditions, boundary conditions, and geometry, based on the transport equations. In this process, the transport coefficients and source terms required for calculating Eqs. (1) and (2) are computed through costly external models. In many cases, the total elapsed time for the predictive simulations depends on how frequently external models are called serially. In the case of using FDM, the discretized PDE is sequentially calculated at each time step starting from the initial condition and repeating until the final step. This typically requires $O(10^5)$ serial calls of the external models for a single tokamak discharge. On the other hand, predictive simulations using PINN start with an initialized functional neural network of θ and gradually update it to satisfy the boundary conditions, initial conditions, and PDEs. External models are called at each update.

For external models to calculate the transport coefficients and source profiles, costly and theory-based codes such as TGLF [12] and NUBEAM [34] can be used. However, in this study, the Bohm/gyro-Bohm transport model [35] was used and the source terms were prescribed as $P_e = 0.1 (1 - \rho^2)$ MW/m³ and $S_e = 4 \times 10^{19}$ m⁻³ s⁻¹, in order for reproducible comparisons at desktop-level computers. For the plasma geometry, we used a sample magneto-hydrodynamic equilibrium from discharge 26381 in the KSTAR tokamak [29], as shown in Fig. 3a. ICs of the plasma profiles (u^{IC} in Eq. (7)) were set as parabolic shapes (See the black dashed lines in Fig. 3c–f), and BCs (u^{BC} in Eq. (6)) were set as constants with the boundary values in the initial condition. After implementing the plasma geometry, BC, IC, and PDEs in PINN, the neural network θ is iteratively updated to reduce the objective function, Eq. (8), using the optimization algorithm, L-BFGS [32], which is one of the classical optimizers. In the process of calculating the objective function, the number of sampling points for loss evaluation and the weights for the losses were set to $(N_{sample}, N_{BC}, N_{IC}) = (10^3, 10^2, 10^2)$ and $(w_{PDE}, w_{BC}, w_{IC}) = (10^3, 1, 1)$. The evolution of the losses with respect to the iterations is shown in Fig. 3b.

Fig. 3c–f show the results of the predictive simulation from $t_i = 0$ s to $t_f = 1$ s using PINN. Fig. 3c and d show the spatiotemporal profiles of electron density and temperature calculated by PINN. In Fig. 3e and f, the 1D profiles calculated by PINN (red) at $t = 0.1$ s and 1.0 s are compared to those calculated by FDM (blue). For comparison, the FDM used a Forward-Time-Centered-Space scheme (used in ASTRA [5]) with $\Delta\rho = 0.01$, $\Delta t = 5 \times 10^{-6}$ s intervals ($N_t = (t_f - t_i)/\Delta t = 2 \times 10^5$). Note that the FDM results diverged due to numerical instability when $\Delta t \gtrsim 1 \times 10^{-5}$ s.

As a result, the difference between PINN and FDM results within the domain was within 5%. This is smaller than the differences among particle-only transport simulation results obtained using other different FDM solvers [3]. Additionally, the number of serial calls to the transport model during the PINN calculation was less than 10^3 . ($N_{iter} = 400$), which can reduce the computation time spent on external models by several hundred times compared to FDM. Here, note that we compared cases with steady-state and prescribed sources to simplify the modeling. However, in actual plasmas, the source varies over time, making the plasma response more dynamic, which complicates the problem. While PINN can also be applied to simulate the dynamical evolutions of plasma with time-varying sources, we observed lower convergence in such cases. More detailed analysis on this will be addressed in our future work.

Table 1

The number of calls for external models in FDM and PINN.

	FDM	PINN
Serial calls (~ computing time)	$N_t = O(10^5)$ (time steps)	$N_{iter} = O(10^3)$ (updates)
Parallel calls (~ computing processors)	$N_\rho = O(10^2)$ (spatial grids in ρ)	$N_{sample} = O(10^3)$ (spatiotemporal sample points)

3. Potential advantages of PINN for tokamak transport

3.1. Acceleration by non-chronological problem solving of PINN

One of the significant differences of the PINN-based simulations from the FDM-based ones is that the computation process to obtain the final solution does not proceed in chronological order in the simulation. In FDM, the discretized equations are solved incrementally by advancing Δt at a time, which inevitably requires calculations in temporal order. However, PINN simultaneously updates the profiles across all time points as the optimization is performed. As a result, the number of iterations (or the number of serial calls to external models) required in PINN is not directly proportional to the simulation's total time interval.

Table 1 shows the number of serial and parallel calls needed to obtain the results in Fig. 3 using FDM and PINN methods. The number of serial calls to external models was several orders of magnitude higher for FDM compared to PINN. However, in addition to these serial calls, both FDM and PINN require multiple calls for each iteration; FDM needs multiple calls for multiple ρ grid points (N_ρ), while PINN requires multiple calls for the sampled points in the spatiotemporal domain (N_{sample}). Since these calls are relatively independent of each other, they can be parallelized using multiple processors. For parallel calls, PINN requires a higher number of calls as it is applied across the spatiotemporal domain, compared to FDM, which can only be parallelized in the spatial domain.

While serial calls directly affect the computation time, parallel calls depend on the number of available processors. This means that in an advanced environment with multiple processors capable of parallel computing, such as GPUs, TPUs, or neuromorphic hardware [36], PINN can potentially offer more efficient computations than FDM. Additionally, since PINN updates the profiles across the entire spatiotemporal domain from the beginning, it can provide a rough estimation of the overall profile solution even with just a few iterations. This can be particularly helpful in the context of actual tokamak experiments, where there is limited time between shots and a rough prediction of the entire discharge may be required, even if it is not highly accurate.

For a more quantitative comparison, we examined the FDM and PINN in a univariate one-dimensional transport situation where an exact solution exists. The one-dimensional transport equation with $D = 1$ m²/s is as shown in Eq. (10), and the initial condition and boundary condition are represented in Eq. (11). The exact solution is given by Eq. (12), and the confinement time is $\tau_e = \rho_{boundary}^2/D = 1$ s.

$$\frac{\partial u}{\partial t} - \frac{\partial^2 u}{\partial \rho^2} - \frac{\pi^2}{2} \cos \frac{\pi}{2} \rho = 0 \quad (10)$$

$$u(t=0) = \cos \frac{\pi}{2} \rho, \quad u(|\rho| = \rho_{boundary} = 1) = 0 \quad (11)$$

$$u(\rho, t) = \left(2 - \exp\left(-\frac{\pi^2}{4}t\right)\right) \cos \frac{\pi}{2} \rho \quad (12)$$

In this context, a comparison of the number of serial calls required depending on the length of the time domain (T_{pulse}) is shown in Fig. 4. For FDM (blue), $\Delta t = 0.5(\Delta\rho)^2$ was set [14], and $\Delta\rho$ was adjusted differently in each case so that the mean-squared error (MSE) between the final solution and the exact solution was less than 5×10^{-4} . PINN (red) shows the result of iterations performed until the MSE value was less than 5×10^{-4} . Since the random seed affects the L-BFGS optimization

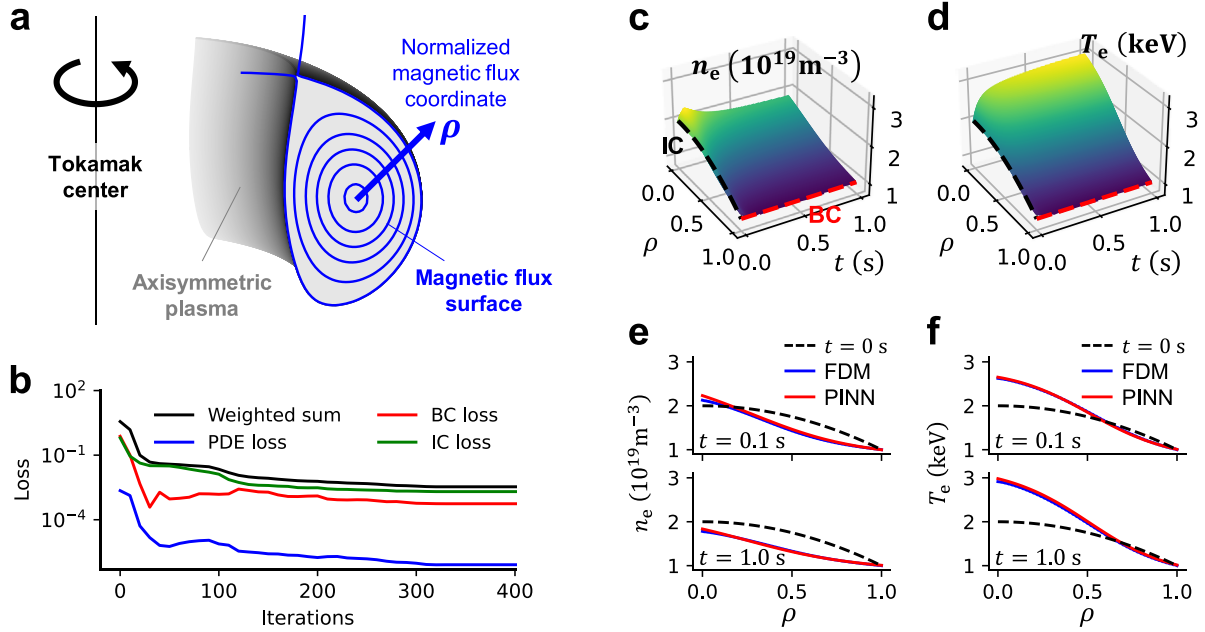


Fig. 3. Predictive transport simulation using PINN in a tokamak. **a**, The example configuration of a tokamak plasma. The three-dimensional plasma geometry is reduced to one-dimension using normalized magnetic flux coordinate, ρ . **b**, The history of the losses during PINN iterations. **c–d**, The electron density (**c**) and temperature (**d**) profiles in spatiotemporal dimensions (ρ, t), simulated using PINN. The initial and boundary conditions are shown in dashed lines. **e–f**, The electron density (**e**) and temperature (**f**) profiles in ρ -space for $t = 0.1$ s and $t = 1.0$ s. The black dashed lines are the initial condition, and blues and reds are the simulation results using FDM and PINN, respectively. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

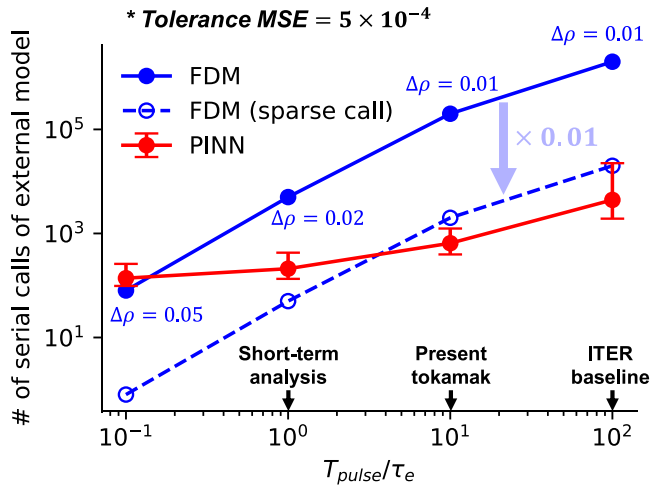


Fig. 4. The number of serial calls for external models with respect to the pulse length of the tokamak discharge. The blue solid line represents the results using the reference FDM, the blue dashed line the results of the FDM with sparse calls (1/100 times fewer calls), and the red line the results using PINN. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

of PINN, the average, minimum, and maximum values of ten different optimizations are shown with errorbars (red) in Fig. 4. The blue dashed line assumes a situation where external models in FDM are called sparsely, only 1/100 as often as in the regular FDM approach. Their values were obtained by dividing the baseline FDM results by 100.

In the range of $T_{pulse}/\tau_e = 10^0 - 10^2$, where typical predictive simulations are performed, PINN required fewer serial calls than FDM, and the difference increased as the target discharge length increased. This suggests that PINN could be more efficient than FDM for performing full-range predictive simulations for long-pulse discharges or future ITER operations. In the $T_{pulse}/\tau_e \gtrsim 10$ region, PINN required fewer calls than even the FDM case with sparsely called external models.

3.2. Grid-free computation of PINN-based transport simulation

In FDM-based predictive simulations for tokamak plasmas, one of the practical issues is the error and numerical instability arising from the discretization of PDEs on finite grids. For example, if we zoom in on the previous FDM results in Fig. 3, we can see a fluctuating gradient as shown in Fig. 5a and b (blue). In many numerical analyses, we need to evaluate plasma stability and bootstrap current, which depend on gradients and higher-order derivatives of plasma profiles. In such situations, strong numerical instabilities can occur, as shown in Fig. 5c, depending on the differencing scheme used in other codes. Here, Fig. 5c shows the bootstrap current, j_{BS} , calculated as

$$j_{BS} = -\epsilon^{1/2} \frac{1}{B_p} \frac{\partial p}{\partial r} \quad (13)$$

where ϵ is the inverse aspect ratio, B_p the poloidal magnetic field, and p the pressure [37]. In high-performance plasmas, the bootstrap current becomes dominant and highly influences plasma equilibrium; for this reason, FDM-based numerical simulations often diverge when the fluctuating current profile locally flips its sign.

In contrast to FDM, which calculates a set of values on discrete grids, PINN finds a function mapping spatiotemporal coordinates and plasma states. This function is composed of differentiable nodes; therefore, exact derivatives, rather than discretization, can be used when calculating PDEs or passing gradients to external models. This frees the method from the numerical instability and errors caused by finite grids. Therefore, PINN may be more suitable for predictive simulations of tokamak plasma that require multiple high-order derivatives. When using profiles calculated with PINN, numerically fluctuating phenomena did not occur, as seen in Fig. 5b and c (red).

Although we focused on the explicit FDM scheme as a baseline, a recent study has shown that using an implicit scheme for FDM is more numerically stable and thus requires fewer mesh sizes [6]. However, because the implicit scheme for tokamak transport requires a special type of root-finding algorithm using matrices to solve Eqs. (1) and (2), it was challenging to make a direct comparison in our study. Nevertheless, more comprehensive quantitative comparisons with various commercial solvers should be conducted in future research.

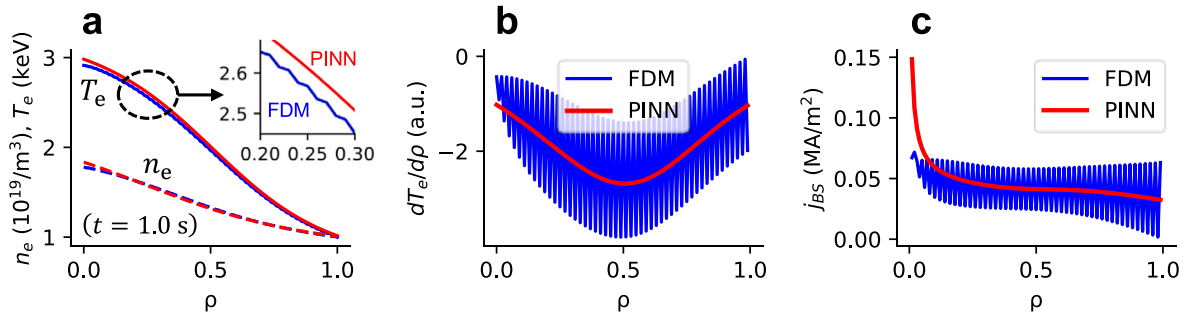


Fig. 5. Comparison of possible numerical instabilities in the transport simulation using FDM and PINN. **a**, Plasma profiles at $t = 1$ s in Fig. 3 and their magnification. Blue represents FDM, red represents PINN, solid lines indicate electron temperature, and dashed lines indicate electron density. **b**, Comparison of electron temperature profile gradients as a function of ρ . **c**, Comparison of the calculated bootstrap current profiles based on these results. While FDM shows the finite grids-induced numerical oscillation, PINN does not. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.3. Using arbitrary spatiotemporal constraints for semi-predictive simulation

FDM requires a complete set of initial profile conditions, which is not highly suitable for diagnostic systems in actual experiments. In real experiments, diagnostic systems may (i) only allow measurements at limited locations, (ii) not be valid under specific conditions such as the ramp-up phase, or (iii) only allow temporally sparse measurements. For example, in cases where the neutral beam is incident in a blip shape (being deliberately flashed on and off every few seconds for diagnostics.), ion temperature measurements through charge exchange spectroscopy (CES) are possible only at a single point every few seconds. In these cases, we need to guess the profiles or interpolate from sparse signals to obtain a complete profile set, which loses diagnostic fidelity. Also, FDM has limitations in predicting areas earlier than initial measurements, such as the early ramp-up phase.

On the other hand, PINN does not have constraints on the spatiotemporal constraints used in Eqs. (6) and (7). Not only the initial conditions but also intermediate states can be used as constraints, and more practical initial and boundary conditions based on finite and sparse measurements can be set [38].

Moreover, we know that predictions through simulations are not completely accurate, and indeed, actual plasmas can behave differently under the same discharge conditions due to unmeasurable factors. In such cases, using PINN, we can perform a “semi-predictive” simulation with corrections that reflect actual experimental tendencies by adding just a few spatiotemporal constraints based on experiments. These additional constraints can be measurements from previous discharges, results from other simulation methods, or constraints based on expert knowledge.

Fig. 6 demonstrates a semi-predictive simulation incorporating arbitrary spatiotemporal constraints that can be obtained from measurements. Not only the initial condition at $t = 0$ s (temporal constraint 1, TC 1), but also additional temporal constraints (TC 2) of three radial points that are measured in the core region at $t = 1$ s (see Fig. 6e and f) are considered for PINN. These additional temporal constraints are applied in the same way as Eq. (7). In comparison, the FDM counterpart cannot account for TC 2 and only uses TC 1 as the initial condition, as shown in Fig. 6c. Since transport equations and coefficient models are not perfect, considering only TC 1 in FDM may result in errors relative to the measurements, TC 2. In such cases, employing a PINN calculation incorporating these measurements enables a semi-predictive simulation that corrects the transport equation. Notably, the PINN result does not merely interpolate between TC 1 and TC 2; instead, it provides a solution that satisfies these constraints while minimizing the error of the given PDE. The importance of additional constraints can be adjusted by increasing or decreasing their weight in Eq. (8) according to the reliability of the TC 2 measurements. We note that when the weight for TC 2 is 0, the situation is equivalent to FDM with TC 1 as the

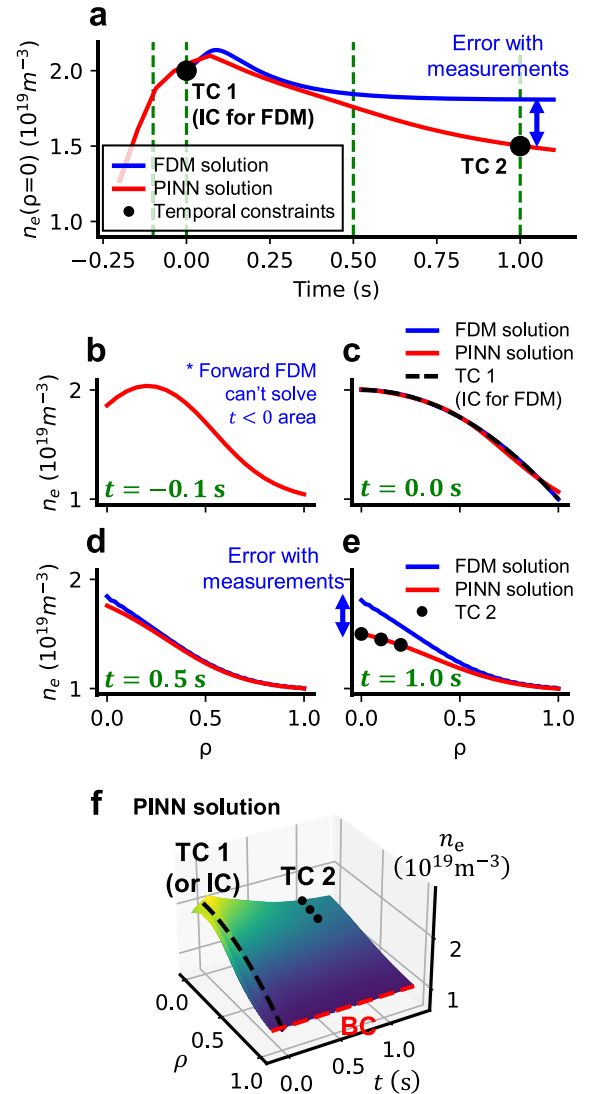


Fig. 6. Semi-predictive simulation considering additional temporal constraint (TC 2) for the correction of PDEs. **a**, Time traces of the core electron density. FDM (blue) can use only the first constraint (TC 1) as the initial condition, which can deviate from the assumed measurements at TC 2. PINN (red) uses both temporal constraints. **b–e**, Electron density profiles in ρ . TC 1 and 2 are also shown in **c** and **e**, while TC 2 is assumed to be three measured values at $t = 1$ s. **f** The PINN solution in spatiotemporal dimensions. The spatiotemporal constraints (TCs and BC) are also shown. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

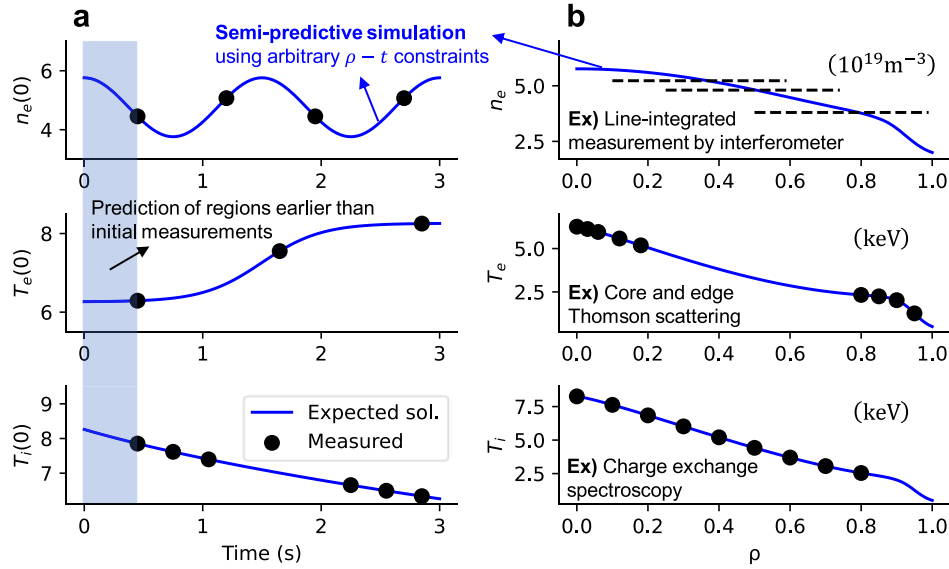


Fig. 7. An example of possible semi-predictive simulation using PINN. a, The measurements and semi-predictive simulation in time. b, The measurements and semi-predictive simulation in ρ . Different diagnostics have different measurement positions and resolutions. Note that these are anticipated illustrations for description, not actual simulation results.

initial condition (fully predictive simulation). In this case, PINN yields a solution almost the same as FDM, as shown in Fig. 3.

Moreover, while FDM has difficulty predicting regions earlier than initial measurements, PINN can estimate prior plasma states ($t < 0$ s) that satisfy the initial condition at $t = 0$ s, as shown in Fig. 6b. In PINN, the initial condition is just one of the possible temporal constraints. This can be utilized for predicting the beginning phase of the discharge beyond the diagnostic validity regime or for predicting surrounding temporal domains from temporally sparse diagnostics. In addition to temporal constraints, spatial constraints and channel-specific constraints can also be arbitrary. Different diagnostics such as Thomson scattering and CES have different measurement locations and resolutions, and even interferometer measures line-integrated values, as illustrated in Fig. 7. For constructing initial conditions in FDM using these measurement values, profile reconstruction and time interpolation are required. In contrast, PINN can preserve the fidelity of the diagnostics by setting arbitrary finite measurement points or integrated values as constraints for optimizations.

3.4. Finding a steady-state solution

Another potential application of PINN is the task of finding steady-state profile solutions. Future fusion reactors may maintain stationary heating and current drive conditions, and thus, we need to estimate the steady-state plasma state. The traditional approach to finding steady-state solutions involves using a time-dependent transport solver under constant heating conditions and running it until the plasma profiles converge [3,39]. Depending on the situation, it can take a long time for the plasma state to converge, and if the numerical settings are not sufficiently fine, the solution might not converge and may oscillate around a certain state.

However, when using PINN, finding a steady-state solution becomes a much simpler problem compared to the time-dependent situation described in Section 2. For this specific task, a few reformulations are necessary. First, in Eqs. (1) and (2), the time-dependent terms should be set to zero, and the neural network input should be changed from (ρ, t) to (ρ) . Additionally, in finding the steady-state solution, the initial condition loss becomes unnecessary, leaving only the first two terms in Eq. (8). In fact, from the perspective of PINN, the equations are simplified, the neural network becomes lighter, and the loss function becomes smoother, significantly increasing training efficiency, as shown in the new Fig. 8.

Fig. 8a shows the loss history for the task of finding a steady-state solution, and it decreases much faster and to a lower level compared to the loss level in Section 2 (shown in gray). The obtained profile (shown in red in Fig. 8b) matches the baseline solution, which was converged using FDM over a sufficiently long time, within an error margin of 5%.

4. Remained challenges of PINN for tokamak transport

In the previous sections, we have mainly discussed the advantages of using a PINN-based tokamak transport solver. However, as PINN is a relatively new technique that has not been widely applied in the fusion community, there are several weaknesses. The first drawback is the inherent issue in deep learning techniques related to the uncertainty and non-convergence caused by random seeds. The sampling of points for calculating the losses in Eqs. (5)–(7) and the neural network optimization algorithms, such as L-BFGS, are influenced by random seeds. As training progresses, the response to these seeds accumulates, potentially leading to non-convergence or trapping in local minima. Fig. 9 shows the results of training the same problem (described in Section 2) with five different seed numbers. As shown in Fig. 9a, the convergence speed towards the final solution varies, and in some cases, training becomes trapped in local minima (magenta). Consequently, the resulting profiles and their temporal evolution also differ, as illustrated in Fig. 9b and c, respectively. Particularly, for the less converged cases (blue and magenta), there are visually significant errors compared to the FDM baseline solution shown in black. Although it is possible to train multiple times with different seed numbers and select the solution with the minimum loss, this approach reduces efficiency.

Secondly, the weights for the IC, BC, and PDE losses shown in Eq. (8) are free parameters, and their selection impacts the outcomes of PINN training. Achieving balanced optimization among different loss functions requires some level of empirical intuition from the researcher. Recently, meta-learning approaches that add an additional loop for hyperparameter optimization have been developed to minimize subjective intervention and ensure more robust performance [40].

Finally, this study focuses on relatively simple low-confinement mode profiles. However, high-confinement mode plasmas are characterized by steep pedestals at the edge and non-monotonic current profiles, which significantly increase the complexity of the problem. Furthermore, solving for the diffusion of ion temperature and magnetic flux, in addition to the electron density and temperature covered in this study, requires more effective optimization algorithms. We hope that the ongoing advancements in machine learning will lead to the development of more efficient PINN-based solvers.

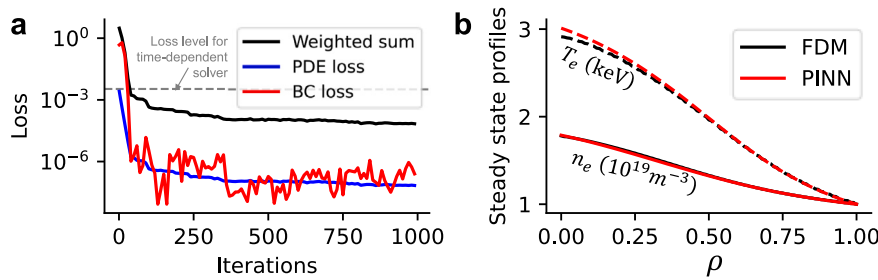


Fig. 8. Finding a steady-state solution using PINN. **a**, The history of the losses during PINN iterations. **b**, The obtained steady-state profiles in red, with the electron density (solid) and temperature (dashed). The baseline solutions using FDM are also shown in black. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

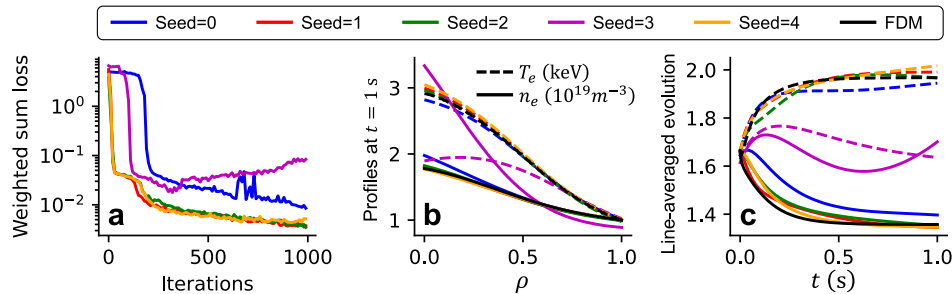


Fig. 9. Random seed effects on the PINN-based transport solver. **a**, The history of the weighted sum losses during PINN iterations. Each color indicates a different random seed number. **b**, Comparison of the electron temperature (dashed) and density (solid) profiles at $t = 1$ s. **c**, Comparison of the time evolution of the line-averaged electron temperature (dashed) and density (solid). In **b** and **c**, the FDM results are shown in black, as a baseline. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5. Discussion and summary

In this study, we introduce a new scheme for the tokamak transport solver based on PINN, a neural computing approach that optimizes a function mapping coordinates and states, as opposed to FDM which calculates PDEs over time on finite grids. The relationship between FDM and PINN is analogous to that of Newtonian and Lagrangian mechanics. While the former simulates nature based on causality over time, the latter finds solutions that minimize a scalar quantity (objective function or action).

Computing transport equations with PINN has three potential benefits:

- Since calculations are not performed sequentially in time in simulations, the number of serial calls to costly external models required for solution convergence is reduced by several orders of magnitude than FDM. Furthermore, the higher parallelizable degree allows for synergy with parallel computing and the future neuromorphic hardware [36].
- The absence of finite grids eliminates numerical instability arising from their use. Additionally, exact derivatives, rather than finite differences, can be used in further detailed analyses, providing higher-fidelity information.
- Arbitrary spatiotemporal constraints can be applied, rather than just initial or boundary conditions. This enables the prediction of plasma states earlier than initial measurements and the “semi-predictive” simulation considering additional temporal constraints. In particular, as shown in Fig. 7, different diagnostics in a tokamak have different measurement positions and resolutions. PINN allows for more experiment-relevant simulations by applying the raw values of actual measurements as constraints, rather than fitted or interpolated profiles required for FDM.

Along with these advantages, there are still challenges that must be overcome. Major caveats are uncertainty due to random seed, and the need for the researcher’s empirical intuition to determine hyperparameters. Additionally, a more effective optimization algorithm is needed to simulate advanced plasma operations such as high-confinement

mode. These hurdles require closer collaboration for physicists with researchers in the core machine learning field.

Interpretive simulations that estimate transport coefficients from measurements (the inverse problem) are also possible [22]. Solving the inverse problem using PINNs to calculate the trajectory of coefficients or sources that satisfy a given plasma state can also be applied to the optimization of tokamak operation scenarios. To operate high-performance plasmas stably in future fusion reactors, it is essential to optimize operating scenarios. While scenario optimization using FDM-based predictive simulation requires numerous independent simulations for various discharge trajectories under different conditions, using PINNs turns it into a problem that only requires a single convergence by adding the optimizing target to the objective function [41]. This may be a potential future work using PINNs.

CRedit authorship contribution statement

J. Seo: Conceptualization, Data curation, Project administration, Supervision, Writing – original draft. **I.H. Kim:** Validation, Visualization, Writing – review & editing. **H. Nam:** Validation, Visualization, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the National Research Foundation of Korea (NRF) funded by the Korea government. (Ministry of Science and ICT) (RS-2024-00346024 and RS-2024-00409564)

References

- [1] M. Shimada, et al., Progress in the ITER physics basis - Chapter 1: Overview and summary, *Nucl. Fusion* 47 (6) (2007) S1, <http://dx.doi.org/10.1088/0029-5515/47/6/S01>.
- [2] G.V. Pereverzev, P. Yushmanov, ASTRA. Automated system for transport analysis in a tokamak, 2002, https://pure.mpg.de/rest/items/item_2138238/component/file_2138237/content.
- [3] Y.-S. Na, et al., On benchmarking of simulations of particle transport in ITER, *Nucl. Fusion* 59 (7) (2019) 076026, <http://dx.doi.org/10.1088/1741-4326/ab15e0>.
- [4] J. Citrin, et al., TORAX: A Fast and Differentiable Tokamak Transport Simulator in JAX, arXiv preprint arXiv:2406.06718, 2024, <https://arxiv.org/abs/2406.06718>.
- [5] R. Schramm, et al., Development and application of a predictive model for advanced tokamak scenario design, *Nucl. Fusion* 64 (3) (2024) 036013, <http://dx.doi.org/10.1088/1741-4326/ad2062>.
- [6] J. Park, et al., An efficient transport solver for tokamak plasmas, *Comput. Phys. Comm.* (ISSN: 0010-4655) 214 (2017) 1–5, <https://www.sciencedirect.com/science/article/pii/S0010465516303976>.
- [7] F. Felici, et al., Real-time-capable prediction of temperature and density profiles in a tokamak using RAPTOR and a first-principle-based transport model, *Nucl. Fusion* 58 (9) (2018) 096006, <http://dx.doi.org/10.1088/1741-4326/aac8f0>.
- [8] B.A. Grierson, et al., Orchestrating TRANSP simulations for interpretative and predictive tokamak modeling with OMFIT, *Fusion Sci. Technol.* 74 (1–2) (2018) 101–115, <http://dx.doi.org/10.1080/15361055.2017.1398585>.
- [9] C.Y. Lee, et al., Development of integrated suite of codes and its validation on KSTAR, *Nucl. Fusion* 61 (9) (2021) 096020, <http://dx.doi.org/10.1088/1741-4326/ac1690>.
- [10] T.S. Hahm, P.H. Diamond, Mesoscopic transport events and the breakdown of Fick's law for turbulent fluxes, *J. Korean Phys. Soc.* (ISSN: 1976-8524) 73 (6) (2018) 747–792, <http://dx.doi.org/10.3938/jkps.73.747>.
- [11] J.-M. Kwon, L. Qi, S. Yi, T. Hahm, ITG–TEM turbulence simulation with bounce-averaged kinetic electrons in tokamak geometry, *Comput. Phys. Comm.* (ISSN: 0010-4655) 215 (2017) 81–90, <https://www.sciencedirect.com/science/article/pii/S0010465517300619>.
- [12] G.M. Staebler, J.E. Kinsey, R.E. Waltz, A theory-based transport model with comprehensive physics, *Phys. Plasmas* (ISSN: 1070-664X) 14 (5) (2007) 055909, <http://dx.doi.org/10.1063/1.2436852>.
- [13] C. Bourdelle, et al., Core turbulent transport in tokamak plasmas: bridging theory and experiment with QuaLiKiz, *Plasma Phys. Control. Fusion* 58 (1) (2015) 014036, <http://dx.doi.org/10.1088/0741-3335/58/1/014036>.
- [14] J. Lee, M.S. Jeong, Stability Of Finite Difference Schemes on the Diffusion Equation with Discontinuous Coefficients, Massachusetts Institute of Technology, Cambridge, 2017, <https://math.mit.edu/research/highschool/rsi/documents/2017Lee.pdf>.
- [15] J. Seo, et al., Feedforward beta control in the KSTAR tokamak by deep reinforcement learning, *Nucl. Fusion* 61 (10) (2021) 106010, <http://dx.doi.org/10.1088/1741-4326/ac121b>.
- [16] J. Seo, et al., Development of an operation trajectory design algorithm for control of multiple 0D parameters using deep reinforcement learning in KSTAR, *Nucl. Fusion* 62 (8) (2022) 086049, <http://dx.doi.org/10.1088/1741-4326/ac79be>.
- [17] J. Degraeve, et al., Magnetic control of tokamak plasmas through deep reinforcement learning, *Nature* (ISSN: 1474-4687) 602 (7897) (2022) 414–419, <http://dx.doi.org/10.1038/s41586-021-04301-9>.
- [18] J. Seo, et al., Avoiding fusion plasma tearing instability with deep reinforcement learning, *Nature* (ISSN: 1474-4687) 626 (8000) (2024) 746–751, <http://dx.doi.org/10.1038/s41586-024-07024-9>.
- [19] R. Shousha, et al., Machine learning-based real-time kinetic profile reconstruction in DIII-D, *Nucl. Fusion* 64 (2) (2024) 026006, <http://dx.doi.org/10.1088/1741-4326/ad142f>.
- [20] B.P. van Milligen, V. Tribaldos, J.A. Jiménez, Neural network differential equation and plasma equilibrium solver, *Phys. Rev. Lett.* 75 (1995) 3594–3597, <https://link.aps.org/doi/10.1103/PhysRevLett.75.3594>.
- [21] M. Raissi, P. Perdikaris, G. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* (ISSN: 0021-9991) 378 (2019) 686–707, <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.
- [22] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, Deepxde: A deep learning library for solving differential equations, *SIAM Rev.* 63 (1) (2021) 208–228, <http://dx.doi.org/10.1137/19M1274067>.
- [23] A. Mathews, N. Mandell, M. Francisquez, J.W. Hughes, A. Hakim, Turbulent field fluctuations in gyrokinetic and fluid plasmas, *Phys. Plasmas* (ISSN: 1070-664X) 28 (11) (2021) 112301, <http://dx.doi.org/10.1063/5.0066064>.
- [24] J. Jeon, J. Lee, S.J. Kim, Finite volume method network for the acceleration of unsteady computational fluid dynamics: Non-reacting and reacting flows, *Int. J. Energy Res.* 46 (8) (2022) 10770–10795, <https://onlinelibrary.wiley.com/doi/abs/10.1002/er.7879>.
- [25] D.A. Kaltsas, G.N. Throumoulopoulos, Neural network tokamak equilibria with incompressible flows, *Phys. Plasmas* (ISSN: 1070-664X) 29 (2) (2022) 022506, <http://dx.doi.org/10.1063/5.0073033>.
- [26] S. Joung, et al., GS-DeepNet: mastering tokamak plasma equilibria with deep neural networks and the Grad-Shafranov equation, *Sci. Rep.* (ISSN: 2045-2322) 13 (1) (2023) 15799, <http://dx.doi.org/10.1038/s41598-023-42991-5>.
- [27] G. Pereverzev, G. Corrigan, Stable numeric scheme for diffusion equation with a stiff transport, *Comput. Phys. Comm.* (ISSN: 0010-4655) 179 (8) (2008) 579–585, <https://www.sciencedirect.com/science/article/pii/S001046550800218X>.
- [28] S. Jardin, G. Bateman, G. Hammett, L. Ku, On 1D diffusion problems with a gradient-dependent diffusion coefficient, *J. Comput. Phys.* (ISSN: 0021-9991) 227 (20) (2008) 8769–8775, <https://www.sciencedirect.com/science/article/pii/S0021999108003616>.
- [29] Y.-S. Na, et al., Observation of a new type of self-generated current in magnetized plasmas, *Nature Commun.* (ISSN: 2041-1723) 13 (1) (2022) 6477, <http://dx.doi.org/10.1038/s41467-022-34092-0>.
- [30] Martín Abadi, et al., TensorFlow: Large-scale machine learning on heterogeneous systems, 2015, <https://www.tensorflow.org/>, Software available from tensorflow.org.
- [31] P. Virtanen, et al., SciPy 1.0: Fundamental algorithms for scientific computing in python, *Nat. Methods* 17 (2020) 261–272, <http://dx.doi.org/10.1038/s41592-019-0686-2>.
- [32] D.C. Liu, J. Nocedal, On the limited memory BFGS method for large scale optimization, *Math. Program.* (ISSN: 1436-4646) 45 (1) (1989) 503–528, <http://dx.doi.org/10.1007/BF01589116>.
- [33] J. Seo, Tokpin, 2024, <https://github.com/jaem-seo/TokPin>.
- [34] A. Pankin, D. McCune, R. Andre, G. Bateman, A. Kritz, The tokamak Monte Carlo fast ion module NUBEAM in the national transport code collaboration library, *Comput. Phys. Comm.* (ISSN: 0010-4655) 159 (3) (2004) 157–184, <https://www.sciencedirect.com/science/article/pii/S0010465504001109>.
- [35] M. Erba, T. Aniel, V. Basiuk, A. Becoulet, X. Litaudon, Validation of a new mixed Bohm/gyro-Bohm model for electron and ion heat transport against the ITER, Tore Supra and START database discharges, *Nucl. Fusion* 38 (7) (1998) 1013, <http://dx.doi.org/10.1088/0029-5515/38/7/305>.
- [36] A. Shahid, M. Mushtaq, A survey comparing specialized hardware and evolution in TPUs for neural networks, in: 2020 IEEE 23rd International Multi-topic Conference, INMIC, 2020, pp. 1–6, <https://ieeexplore.ieee.org/document/9318136>.
- [37] K. Miyamoto, Plasma physics and controlled nuclear fusion, Springer Series on Atomic, Optical, and Plasma Physics, Springer, ISBN: 9783540242178, 2005, <https://books.google.co.kr/books?id=tLq4hzVnN8C>.
- [38] J. Seo, Past rewinding of the fluid dynamics from noisy observation via physics-informed neural computing, *Phys. Rev. E* (2024) in press.
- [39] C. Kessel, et al., Simulation of the hybrid and steady state advanced operating modes in ITER, *Nucl. Fusion* 47 (9) (2007) 1274, <http://dx.doi.org/10.1088/0029-5515/47/9/026>.
- [40] A.F. Psaros, K. Kawaguchi, G.E. Karniadakis, Meta-learning PINN loss functions, *J. Comput. Phys.* (ISSN: 0021-9991) 458 (2022) 111121, <https://www.sciencedirect.com/science/article/pii/S0021999122001838>.
- [41] J. Seo, Solving real-world optimization tasks using physics-informed neural computing, *Sci. Rep.* (ISSN: 2045-2322) 14 (1) (2024) 202, <http://dx.doi.org/10.1038/s41598-023-49977-3>.